

Building a Pega Constellation Application

A Hands-On, Step-by-Step Learning Guide

From class structure and the data model, through case management, views, integrations and decisioning, to the login page and authentication.

Covers: Classes & class hierarchy • DB tables & data model • Case types & stages • Constellation views • Cascading dropdowns • Attachments • Maps • Data transforms • Activities • Decision tables • Connect REST • Service REST • Connect SQL • Data pages • Authentication & login • Security • Reporting • Testing & deployment

Aligned to Pega Infinity '25 (Constellation architecture). Prepared as a self-study workbook.

Contents

1. How to Use This Guide	5
The running example: a "Vehicle Insurance Claim" application	5
Prerequisites	5
A word on Constellation vs. the traditional UI	5
2. Pega and the Constellation Architecture	6
What Pega is	6
The Constellation model: configure, don't customize	6
Current version context	6
The two studios you will live in	6
3. Application Structure: Classes, Class Hierarchy & RuleSets	7
Classes and the class hierarchy	7
The class group (work pool)	7
RuleSets and versions	7
The Application rule	8
4. The Data Model: Properties, Data Objects & Field Types	9
Properties	9
Data objects and field types in App Studio	9
5. Database Tables: Where Your Data Physically Lives	11
How Pega stores data by default	11
Exposing (optimizing) a property to a column	11
The Database Table and Database records	11
Mapping a class to an external table	11
6. Case Management: Case Types, Stages, Steps & Life Cycle	13
The case life cycle model	13
Stage types	13
Routing and SLAs	13

7. Constellation Views: Building the UI Without Sections	15
What a view is	15
Common view templates	15
Extending the design system: DX components	15
8. Cascading (Dependent) Dropdowns	16
The building blocks	16
Step-by-step: Make → Model	16
9. Attachments: Files, Documents & Categories	17
How attachments work	17
Ways to capture attachments	17
10. Maps & Geolocation	18
Approaches to maps in Pega	18
Step-by-step: geocode the incident address	18
11. Data Transforms: The Preferred Way to Manipulate Data	19
What a data transform does	19
pyDefault: model defaults	19
12. Activities: Procedural Logic (Use Sparingly)	21
Anatomy of an activity	21
Frequently used methods	21
When an activity is legitimately the right tool	21
13. Decision Tables & Other Decisioning Rules	23
The family of decision rules	23
Building the claim triage decision table	23
14. Integration – Connect REST (Calling External APIs)	25
Key concepts	25
15. Integration – Service REST (Exposing Pega as an API)	26
Key concepts	26

16. Integration – Connect SQL (Custom Database Queries)	27
When (and when not) to use Connect SQL	27
The pieces	27
17. Data Pages: The Clean Interface to All Data	28
Anatomy	28
Why data pages matter in Constellation	28
18. The Login Page & Authentication	29
Authentication services	29
Authentication for Constellation specifically	29
Customising the login experience	29
Operators, access groups and the login chain	30
19. Authorization & Security: Roles, Privileges & Access Control	31
The building blocks	31
Row and column security (ABAC)	31
20. Reporting: Report Definitions & Insights	32
Report definition essentials	32
21. Quality & Delivery: Guardrails, Testing & Deployment	33
Guardrails and the compliance score	33
Testing	33
Packaging and moving code	33
22. Putting It All Together: The ClaimsApp Build Order	34
Where to learn more (official sources)	34

1. How to Use This Guide

This workbook walks you through building one realistic Pega Constellation application end to end. Each chapter introduces a rule type or feature, explains why it exists, and then gives you concrete click-by-click steps you can reproduce in your own Pega environment.

The running example: a "Vehicle Insurance Claim" application

Throughout the guide we build a single application called **ClaimsApp** for a fictional insurer, **AcmelInsure**. A customer files a vehicle insurance claim; the claim is triaged, an assessor is assigned, repair estimates are gathered from an external service, a decision is made on payout, and the claim is resolved. This one thread lets every feature in the guide connect to a real purpose rather than sitting in isolation.

Prerequisites

- Access to a Pega Infinity environment (Pega Platform '24 or '25). A free **Pega Cloud personal edition** or a partner/enterprise sandbox works well.
- The **App Studio** and **Dev Studio** portals. Constellation development is *App Studio first*; you drop into Dev Studio only for lower-level rules (activities, connectors, SQL, security).
- An operator ID with an access group that grants author privileges (for example a role derived from **PegaRULES:SysAdm4** in a sandbox).
- Basic familiarity with REST, JSON and SQL concepts — this guide explains the Pega side, not those standards.

How the numbered steps work

Wherever you see the numbered teal badges, those are actions to perform in the product. Menu paths are written as **Portal ' Menu ' Item**. Rule form field names are shown in **bold**.

A word on Constellation vs. the traditional UI

Pega has two UI architectures. The older **traditional** architecture builds screens from *harnesses*, *sections* and *flow actions* that you hand-craft. The newer **Constellation** architecture is *model-driven*: you configure the data model and case life cycle, and Pega generates a prescriptive, React-based, single-page UI for you. This guide is written for Constellation, which is now Pega's mainstream, recommended architecture for new applications.

2. Pega and the Constellation Architecture

What Pega is

Pega is a low-code platform for building enterprise **workflow (case management)** and **customer engagement** applications. You express business logic as **rules** — reusable, versioned, auditable configuration objects — rather than as hand-written code. Rules live in **RuleSets**, are organized by **class**, and are resolved at run time based on the logged-in user's context.

The Constellation model: configure, don't customize

With the traditional architecture you *customize* pixels. With Constellation you *configure* a model and Pega generates the experience. The mental shift is the single most important thing to internalize.

- **Design system** — a library of pre-built, React-based UI components and patterns. You choose templates; you do not draw layouts by hand.
- **Constellation architecture** — a client-side, single-page application (SPA). The browser loads a React shell and renders UI from metadata, so there are no server-rendered HTML pages and no manual screen refreshes.
- **Constellation DX API** — a set of model-driven REST endpoints ("DX API v2") that return data *and* UI metadata separately. The same endpoints power both Pega's own UI and any custom React/Angular/Vue front end.

Center-out™ business architecture

Constellation embodies Pega's Center-out approach: you model the **case** (the unit of work) and the **data** in the middle, then project outward to *channels* (web, mobile, API) and inward to *systems of record*. Build the middle once; reuse it everywhere.

Current version context

At the time of writing, the latest release line is **Pega Infinity '25** (version 25.1, first released September 2025, with 25.1.2 in February 2026). Infinity '24 (24.1 / 24.2) is still widely used. Everything in this guide applies across '24 and '25; where a capability is version-sensitive it is noted.

The two studios you will live in

Studio	Who uses it	What you build there
App Studio	Citizen & low-code developers	Case types, data model, views, personas, channels, data objects, most day-to-day configuration.
Dev Studio	System architects	Activities, data transforms, connectors (REST/SOAP/SQL), service rules, security rules, class & RuleSet administration, decision rules, database records.

Everything is a rule

A case type, a property, a view, a data transform, an activity, a decision table, a connector, an access group — all of these are **rule instances** stored in the PegaRULES database. Understanding "rule resolution" (how Pega picks which version of a rule to run for a given user) is the foundation for everything that follows.

3. Application Structure: Classes, Class Hierarchy & RuleSets

Before any screen exists, Pega needs to know *where* your rules live. That "where" is defined by the class hierarchy and the RuleSet stack.

Classes and the class hierarchy

A **class** is a container that groups rules and defines the structure of the objects it describes (its properties). Classes are arranged in a hierarchy and **inherit** rules from their parents, which is how reuse works in Pega. Every class ultimately descends from **@baseclass**.

Two kinds of inheritance

- **Pattern inheritance** — follows the dotted naming of the class (e.g. AcmeInsure-ClaimsApp-Work-Claim looks up through AcmeInsure-ClaimsApp-Work, then AcmeInsure-ClaimsApp-, etc.).
- **Direct inheritance** — an explicit parent you set on the class form (e.g. a Work class directly inherits from Work-, giving it all standard case behaviour).

The layered class structure of a Pega application

Layer	Example class	Purpose
Organization	AcmeInsure-	Top of your enterprise; org-wide reusable rules.
Division / Unit	AcmeInsure-Claims-	Optional grouping for a business division.
Application (Work)	AcmeInsure-ClaimsApp-Work-	Abstract parent of your case types.
Case type (concrete)	AcmeInsure-ClaimsApp-Work-Claim	A concrete Work class – an actual case.
Data	AcmeInsure-ClaimsApp-Data-Vehicle	Concrete Data class – backs a data object.
Integration	AcmeInsure-ClaimsApp-Int-...	Classes generated for connectors / integrations.

Abstract vs. concrete classes

An **abstract** class (name ends in a hyphen, e.g. ...-Work-) cannot have object instances saved against it; it exists to hold shared rules. A **concrete** class (e.g. ...-Work-Claim) can be instantiated — a real claim is stored as an instance of it.

The class group (work pool)

A **class group** groups related concrete Work classes so their instances share one database table. Your application's work pool (e.g. AcmeInsure-ClaimsApp-Work) is the class group; all case types created under it store their instances in the same pc_work-style table by default.

RuleSets and versions

A **RuleSet** is a named collection of rules; a **RuleSet version** (like ClaimsApp:01-01-01) is how you package and lock changes. The ordered list of RuleSets available to a user is the **RuleSet stack**, and it

drives rule resolution together with the class hierarchy.

- **Major.Minor.Patch** versioning (e.g. 01-02-05). Higher versions override lower ones during resolution.
- **Locking** a version freezes it so it can be safely migrated; new work continues in a higher version.
- Application-layer RuleSets sit *above* Pega's built-in RuleSets, so your rules override platform defaults.

The Application rule

The **Application** rule ties it together: it lists the RuleSet versions in scope, the built-on application (usually the Pega platform application), the work pools, and the case types. When you create a new application with the **New Application wizard**, Pega generates the organization class, application RuleSet, work pool, an Application rule, and a starter access group for you.

Create the ClaimsApp application

- 1 Log in to Pega and open **App Studio**. From the app switcher choose **New Application**.
- 2 Select **Build a new app from scratch** (or start from a Blueprint). Enter the application name **ClaimsApp** and organization **AcmeInsure**.
- 3 Accept or edit the generated **organization**, **RuleSet** (ClaimsApp) and **work pool** (AcmeInsure-ClaimsApp-Work) names. Confirm the Constellation (Cosmos React) UI is selected – it is the default for new apps.
- 4 Finish the wizard. Pega provisions the class hierarchy, RuleSet stack, Application rule and a sample access group.
- 5 Switch to **Dev Studio** once (App menu ' Dev Studio) and open **Records ' SysAdmin ' Application** to review what was generated – this cements the mental model before you build features.

Naming convention that scales

Keep implementation-specific case types in . . . -Work - and reusable data in . . . -Data -. Resist putting business logic in the organization layer unless it is truly enterprise-wide – it is hard to unwind later.

4. The Data Model: Properties, Data Objects & Field Types

In Constellation the data model is the star of the show. Because the UI is generated from the model, getting fields and their types right is what actually shapes your screens.

Properties

A **property** is a single piece of data belonging to a class (e.g. `ClaimAmount`, `VehicleRegistration`). Each property has a **type** and a **mode**.

Property modes (structure)

Mode	Holds	Example
Single Value	One scalar value	<code>ClaimAmount</code> (Decimal)
Value List	Ordered list of scalars	<code>DamagePhotos</code> (list of text)
Value Group	Keyed set of scalars	<code>TranslatedLabels("en")</code>
Page	One embedded object (a class)	<code>.Vehicle</code> (a Data-Vehicle page)
Page List	Ordered list of embedded objects	<code>.LineItems()</code> repair line items
Page Group	Keyed set of embedded objects	<code>.Addresses("home")</code>

Data objects and field types in App Studio

In App Studio you rarely say "property"; you add **fields** to a **data object** or a case type. A **data object** is App Studio's friendly name for a **Data- class** plus its data model and default views. Behind the scenes every field is still a property.

App Studio field type	Underlying property type	Notes
Text (single line / paragraph)	Text	Most common; add validation as needed.
Integer / Decimal / Currency	Integer / Decimal	Currency shows a symbol; still decimal underneath.
Date & time / Date only	DateTime / Date	Stored in ISO/UTC; displayed per locale.
Boolean (checkbox / toggle)	True-False	Great for conditional visibility.
Picklist (dropdown)	Text + local list or data page	Foundation for cascading dropdowns (Ch. 8).
Email / Phone / URL	Text	Built-in format validation.
Attachment / File	Special	Backed by attachment infrastructure (Ch. 9).
Data reference (embed)	Page / Page List	Embeds another data object; enables reuse.

Model the Claim case data

1

In App Studio open **Data ' Data objects** and create a data object **Vehicle** (class `AcmeInsure-ClaimsApp-Data-Vehicle`).

- 2 Add fields to Vehicle: **Registration** (Text), **Make** (Picklist), **Model** (Picklist), **Year** (Integer), **Value** (Currency).
- 3 Create a data object **RepairEstimate** with **Garage** (Text), **LabourCost** (Currency), **PartsCost** (Currency), **TotalCost** (Currency, calculated).
- 4 Open the **Claim** case type ' **Data model** ' tab. Add fields: **ClaimType** (Picklist), **DateOfIncident** (Date), **Description** (Paragraph), **ClaimAmount** (Currency).
- 5 Embed the Vehicle data object into Claim as a **data reference** (single) and RepairEstimate as a **list** (page list) – this creates the .Vehicle page and .RepairEstimates() page list.

Calculated fields – declare expressions

For **TotalCost = LabourCost + PartsCost**, don't write an activity. Use a **Declare Expression** (a declarative rule): open the field and set it to be calculated by an expression. Pega recalculates it automatically whenever an input changes – no On-Change event, no refresh, fully Constellation-friendly.

5. Database Tables: Where Your Data Physically Lives

Pega abstracts the database, but real projects must control storage, performance and integration with external systems. This chapter connects the logical class to the physical table.

How Pega stores data by default

Every concrete Work or Data class maps to a database table. Out of the box, work objects are stored in the **PegaDATA** schema in a BLOB column (pzPVStream) plus a set of **exposed columns**. The BLOB holds the whole object; exposed columns hold individual properties so they can be queried, filtered and reported on efficiently.

Exposing (optimizing) a property to a column

When you need to filter or report on a property, you **optimize** it, which creates a real database column and populates it. This is the single most common performance task in Pega.

Expose ClaimAmount as a column

- 1 In Dev Studio open the property **ClaimAmount** (or right-click a field in App Studio ' **Optimize for reporting**).
- 2 Choose **Actions** ' **Optimize for reporting** . Pega generates the DDL to add the column.
- 3 Review and apply the schema change (in sandboxes you can apply directly; in higher environments a DBA runs the DDL).
- 4 Pega back-fills existing rows and thereafter keeps the column in sync with the BLOB automatically.

Don't over-expose

Every exposed column is written on each save. Expose only what you filter, sort or report on. For large text or rarely queried data, leave it in the BLOB.

The Database Table and Database records

Two administrative records govern physical storage:

- **Data-Admin-DB-Name (Database)** — a named JDBC connection (e.g. PegaDATA, PegaRULES, or an external DB).
- **Data-Admin-DB-Table (Database Table)** — maps a Pega class to a specific database + schema + table. This is how you point a class at a dedicated or external table.

Mapping a class to an external table

Suppose AcmeInsure keeps its vehicle catalogue in an external Postgres/Oracle table. You can surface it as a Pega class without copying data.

Map Data-Vehicle to an external table

- 1 Create (or ask a DBA for) a **Database** record with the JDBC connection to the external schema.
- 2 Run **Dev Studio ' Configure ' Data Model ' Classes & Properties ' Data Model Generator**, or use the **Connect to external database** wizard, to introspect the external table.
- 3 Pega generates a **class**, a **Database Table** mapping, and **properties** matching the columns.
- 4 The class now reads/writes the external table directly – Obj-Browse, report definitions and data pages all work against it.

When to use which storage

Pega-managed table for case work and app-owned data. **External-mapped class** for reading reference/master data that another system owns. **Connect REST/SQL** (Ch. 12–14) when you must call out to a service or run bespoke SQL rather than persist locally.

6. Case Management: Case Types, Stages, Steps & Life Cycle

The case is the heart of a Pega application. A **case type** is a template for a unit of work; a **case** is a running instance that moves through a life cycle from creation to resolution.

The case life cycle model

Constellation uses the **stages-and-steps** life-cycle model. You read it left to right like a customer journey, which makes it reviewable by business stakeholders.

- **Stage** — a major phase (e.g. *Intake* ' *Assessment* ' *Decision* ' *Resolution*).
- **Process** — a sequence of steps within a stage.
- **Step** — a single unit of work: *Collect information* (a form/view), an *Approval*, an *Automation*, a *Decision*, or a call to a sub-process/child case.
- **Assignment** — a step that waits for a human; it lands in a worklist or work queue.
- **Status** — the case's pyStatusWork value (New, Open, Pending-*, Resolved-Completed, Resolved-Rejected).

Stage types

Stage type	Meaning
Primary	Normal forward flow of the case, shown left to right.
Alternate	Off-track path the case can jump to (e.g. 'Escalation').
Resolution	Terminal stage where the case is resolved and closed.

Design the Claim life cycle

- 1 Open the **Claim** case type ' **Life cycle** tab in App Studio.
- 2 Create primary stages: **Intake**, **Assessment**, **Decision**, and a resolution stage **Resolved**.
- 3 In **Intake**, add a **Collect information** step that shows the claim + vehicle fields (a view). Add an **Automation** step that runs a data transform to set defaults.
- 4 In **Assessment**, add an **Approval** step routed to an *Assessor* work queue, and a step that calls the external repair-estimate service (Connect REST, Ch. 12).
- 5 In **Decision**, add a **Decision** step backed by a decision table (Ch. 11) that recommends Approve / Reject / Refer, then an Approval step for a manager.
- 6 In **Resolved**, set the case status to **Resolved-Completed** (or Rejected) and send a confirmation correspondence.

Routing and SLAs

- **Routing** decides who gets an assignment: to the current user, to a specific operator, or to a **work queue** (work basket) using a routing decision.

- **Service Level Agreement (SLA)** rules define goal/deadline times and escalation actions (notify, reassign, raise urgency) when an assignment ages.
- **Case-wide SLA** can be set on the case type; step-level SLAs refine it.

Child cases and case hierarchy

A Claim can spawn a child **RepairApproval** case per garage. Parent and child cases coordinate: the parent can wait for all children to resolve before proceeding. Model children as separate case types and instantiate them with a **Create case** step.

7. Constellation Views: Building the UI Without Sections

In the traditional architecture you built *sections* and *harnesses*. In Constellation you build **views** — model-driven UI made from templates and fields. This is the biggest day-to-day difference.

What a view is

A **view** is a configuration of fields arranged with a **template**. Pega renders it through the design system, so it is automatically responsive, accessible and consistent. You do not place pixels; you pick a template and assign fields and components to its regions.

Common view templates

Template	Use for
One column / Two column	Standard data-entry forms in a step.
Details	Read-only summary of case fields (case summary panels).
List / Table	A page list rendered as an editable or read-only table.
Field group (single) / (list)	Embedded data object shown inline or as a repeating grid.
Page / Tab / Accordion	Grouping larger forms into navigable regions.

Build the Intake data-entry view

- 1 Open the **Collect information** step in the Intake stage and click **Configure view**.
- 2 Choose a **Two column** template. Drag in **ClaimType**, **DateOfIncident**, **Description**, **ClaimAmount**.
- 3 Add the embedded **Vehicle** field group so registration/make/model appear inline.
- 4 Set **required** and **visibility** conditions per field using **when** rules or expressions (e.g. show *Other claim type detail* only when ClaimType = Other).
- 5 Preview using the built-in **Preview** (or the Live UI/DX runtime) to see the generated responsive form.

Dynamic UI is automatic

Constellation forms are **reactive**: calculated fields, conditional visibility and dependent picklists update on their own from the data model and declarative rules. You should *not* wire On-Change actions or manual 'refresh section' calls — that traditional pattern is both unnecessary and discouraged in Constellation.

Extending the design system: DX components

When a template cannot express a needed UI, you use a **Constellation DX Component** — a custom React component that still lives inside the Constellation runtime and guardrails. Pega ships a component library (and a GitHub gallery) you can adopt or adapt. This is the sanctioned way to add bespoke UI (maps, custom charts, specialized inputs) without abandoning the model-driven approach.

8. Cascading (Dependent) Dropdowns

A classic requirement: pick a **Make**, and the **Model** dropdown shows only models for that make. In Constellation this is done declaratively with parameterized data pages — not On-Change events.

The building blocks

- A **source data page** for each list. The dependent list's data page takes the parent value as a **parameter**.
- A field configured as a **dropdown/picklist** whose **data source** is that data page.
- A declared **dependency**: the child field references the parent field's value as the data page parameter, so Constellation re-sources the child list automatically when the parent changes.

Step-by-step: Make → Model

Create the parameterized data pages

- 1 Create data page **D_VehicleMakes** (list) sourced from a report definition or lookup on Data-Vehicle-Make.
- 2 Create data page **D_VehicleModels** (list) and add a **parameter** named **MakeID**.
- 3 Source D_VehicleModels so its query/filter uses param.MakeID (e.g. a report definition filtered by Make = param.MakeID, or a Connect REST call passing MakeID).

Wire the fields in the view

- 1 In the view, set the **Make** field's list source to **D_VehicleMakes**.
- 2 Set the **Model** field's list source to **D_VehicleModels** and map its parameter **MakeID** to .Vehicle.Make (the value the user picked).
- 3 Save. When the user changes Make, Constellation detects the parameter dependency and re-runs D_VehicleModels with the new MakeID — the Model list refreshes on its own.

Three or more levels

Chain the same pattern: Country → State → City. Each level's data page takes the level above as a parameter. Keep each data page read-only and cached appropriately (see Ch. 15) so repeated lookups stay fast.

Reference fields as cards (Infinity '25)

From Infinity '25 you can render reference (single/multi-select) fields as **cards** instead of plain dropdowns — useful when each option needs a richer visual. The dependency mechanics are the same.

9. Attachments: Files, Documents & Categories

Insurance claims need photos, invoices and police reports. Pega has first-class attachment infrastructure that handles storage, security and versioning for you.

How attachments work

An attachment is stored as a **Data-WorkAttach-*** instance linked to the case. Each attachment belongs to an **attachment category**, which controls *who can view, edit or delete* it via privileges. Files can live in the Pega database or in configured cloud storage (e.g. a repository / S3-backed store).

Ways to capture attachments

Mechanism	When to use
File / Attachment field	Let the user upload a file directly in a view (Constellation field type).
Attach content step	A dedicated life-cycle step prompting the user to attach files/links/notes.
Attachment category	Governs security & grouping (e.g. 'Damage photo', 'Invoice', 'Police report').
Programmatic	Attach files via API/activity (e.g. attach a generated PDF correspondence).

Add secured photo attachments to the Claim

- 1 In Dev Studio create an **Attachment Category** named **DamagePhoto** under your work class.
- 2 On the category, set **view / edit / delete privileges** so only the claimant and assessors manage photos.
- 3 In App Studio, in the Intake view, add a **File** field (or an **Attach content** step) and bind it to the **DamagePhoto** category.
- 4 Restrict allowed file types and size on the field (e.g. JPG/PNG/PDF, max 10 MB).
- 5 Verify uploaded files appear in the case's **Attachments** utility and honour the category's security.

Standard attachment properties

Attachment metadata is exposed via standard properties such as `pxAttachName`, `pyAttachTitle` and category. Use them in reports ('claims missing a police report') and in decisioning without cracking open the file itself.

10. Maps & Geolocation

Claims happen somewhere. Capturing an incident location on a map, or geocoding an address, is a common requirement met by combining a mapping service with Constellation.

Approaches to maps in Pega

- **Geolocation capture** — mobile channels can capture the device's latitude/longitude into properties; store them as Decimal fields on the case.
- **Map DX component** — render an interactive map inside a Constellation view using a custom DX component that wraps a mapping SDK (e.g. Google Maps / Leaflet). This keeps the map inside the design-system runtime.
- **Geocoding via Connect REST** — call a maps API (e.g. Google Geocoding) to turn an address into coordinates, or coordinates into an address, then store the result on the case.

Step-by-step: geocode the incident address

Address to map pin

- 1 Add fields to the Claim: **IncidentAddress** (Text), **Latitude** (Decimal), **Longitude** (Decimal).
- 2 Create a **Connect REST** integration to the mapping provider's geocoding endpoint (see Ch. 12), passing the address and returning lat/long JSON.
- 3 Wrap the connector in a **data page** D_Geocode parameterized by address.
- 4 In an **Automation** step (or a post-processing data transform), call D_Geocode and copy lat/long onto the case.
- 5 Add a **Map DX component** to the case view, bound to Latitude/Longitude, to display the incident pin. (Obtain/adapt the component from Pega's component gallery.)

API keys & secrets

Never hard-code a maps API key in a view or activity. Store it in an **authentication profile** / secrets store and reference it from the Connect REST rule, so keys are managed and rotatable.

11. Data Transforms: The Preferred Way to Manipulate Data

A **data transform** declaratively maps and sets data on the clipboard. In Constellation-era Pega it is the *first choice* for data manipulation — preferred over activities wherever possible.

What a data transform does

A data transform runs a list of **actions** against a source and target page. It is easy to read, guardrail-compliant, and reusable. Common actions:

Action	Effect
Set	Assign a value/expression to a target property.
Update Page	Set multiple properties on an embedded page.
Append to / Append and map to	Add an item to a page list (optionally applying another transform).
Apply Data Transform	Call another data transform (composition/reuse).
For Each Page In	Iterate a page list and act on each element.
Remove / Append	List maintenance.

pyDefault: model defaults

Every case type has a **pyDefault** data transform that runs at creation. Use it to set sensible starting values — default status, SLA-related dates, initial ClaimType, etc. — instead of writing initialization code.

Create a SetClaimDefaults data transform

- 1 In Dev Studio 'Records' > **Data Model** > **Data Transform** > **Create**. Name it **SetClaimDefaults** in the Claim work class.
- 2 Add actions: **Set** .ClaimType = "Vehicle"; **Set** .DateLogged = @CurrentDateTime(); **Set** .Priority = "Normal".
- 3 Add **Apply Data Transform** to call a reusable **MapClaimant** transform that copies the logged-in operator's details onto the case.
- 4 Call SetClaimDefaults from **pyDefault** (or reference it in an Automation step) so it runs on new claims.

Data Transform: SetClaimDefaults (target = the Claim page)

Action	Target	Source / Expression
-----	-----	-----
Set	.ClaimType	"Vehicle"
Set	.DateLogged	@CurrentDateTime()
Set	.Priority	"Normal"
Update Page	.Claimant	(map operator name/email)
For Each	.RepairEstimates	(init .TotalCost = 0 each)

Reading of a simple data transform. Note: declarative, top-to-bottom, no Java.

Prefer data transform over Property-Set-in-activity

A guardrail warning fires when you use **Property-Set** steps in an activity to do work a data transform could do. Move that logic into an **Apply-DataTransform**. This both raises your compliance score and keeps the app maintainable.

12. Activities: Procedural Logic (Use Sparingly)

An **activity** is Pega's procedural rule — an ordered list of **steps**, each calling a **method**. Powerful, but modern Pega guidance is to reach for declarative rules first and use activities only when nothing else fits.

Anatomy of an activity

- **Steps** — numbered lines, each with a **method**, a **step page**, parameters, and an optional **when/loop** precondition.
- **Pages & Classes** tab — declares the named pages the activity works with and their classes.
- **Parameters** tab — typed inputs/outputs so the activity is reusable.
- **Activity type** (Security tab) — e.g. Activity, Utility, Connect, Route — affects where it may be called.

Frequently used methods

Method	Purpose
Property-Set	Set a property (prefer a data transform instead).
Apply-DataTransform	Invoke a data transform from an activity.
Page-New / Page-Copy / Page-Remove	Create/clone/remove clipboard pages.
Obj-Open / Obj-Open-By-Handle	Load an instance from the database.
Obj-Save / Obj-Save (Commit)	Persist an instance.
Obj-Browse	Query multiple instances into a page list.
Call / Branch	Invoke another activity.
Connect-REST / Connect-SQL	Invoke an integration connector (see Ch. 12–14).

When an activity is legitimately the right tool

- Orchestrating an integration call and post-processing its response (often wrapped by a data page).
- Complex iteration/branching that declarative rules cannot express cleanly.
- Bulk/batch processing invoked by an agent/job scheduler.

Activity guardrails

Avoid **Java steps** (use functions instead), avoid the **"right now"** commit option (let Pega commit at flow boundaries), and keep activities **modular** (roughly 25 steps or fewer). Set the correct **activity type** so it is only callable in valid contexts.

Create a CalculatePayout activity (when a transform won't do)

- 1 Dev Studio ' **Records** ' **Technical** ' **Activity** ' **Create** → **CalculatePayout** in the Claim class.
- 2 On **Pages & Classes**, declare the pages you use (the primary case page and any data pages).

- 3 Step 1: **Apply-DataTransform** to normalise estimate line items.
- 4 Step 2: a **when**-guarded loop over `.RepairEstimates` summing costs.
- 5 Step 3: **Property-Set** the final `.ClaimAmount` (kept minimal; heavy mapping stays in transforms).
- 6 Set activity type appropriately and check the guardrail warnings panel before saving.

13. Decision Tables & Other Decisioning Rules

Business rules that pick an outcome from inputs belong in **decision rules**, which business users can read and change — not buried in activities.

The family of decision rules

Rule	Best for
Decision Table	Many conditions → one outcome, expressed as rows/columns (spreadsheet-like).
Decision Tree	Nested if/then branching where order and hierarchy matter.
Map Value	Convert one/two inputs into an output via a grid (e.g. score → band).
When	A single reusable boolean condition.
Declare Expression	Auto-calculated property (forward/backward chaining).

Building the claim triage decision table

We want to recommend **Approve**, **Reject** or **Refer** based on claim amount, customer tenure and whether a police report is attached.

Create the RecommendDecision decision table

- 1 Dev Studio 'Records' 'Decision' 'Decision Table' 'Create' → **RecommendDecision** in the Claim class.
- 2 Define **condition columns**: ClaimAmount, CustomerTenureYears, HasPoliceReport.
- 3 Define the **return** property: .Recommendation.
- 4 Enter rows top to bottom; Pega evaluates in order and returns the first matching row ('evaluate all rows' is optional).
- 5 Set the **otherwise** return to **Refer** so nothing falls through unhandled.
- 6 Run the built-in **Run** / test harness with sample inputs to validate before wiring it into the Decision step.

```
Decision Table: RecommendDecision  returns .Recommendation

ClaimAmount  TenureYears  HasPoliceReport  => Recommendation
-----
<= 1000      >= 1           any              => Approve
<= 5000      >= 3           true             => Approve
> 5000       any           any              => Refer
any          < 1           false            => Reject
otherwise                                => Refer
```

Rows are evaluated top-down; the first match wins. 'otherwise' guarantees a result.

Keep decisions out of code

Because a decision table is data, an analyst can adjust thresholds without a developer. Call it from a **Decision** step or a **Declare Expression**, then branch the case flow on the returned recommendation.

14. Integration – Connect REST (Calling External APIs)

Connect REST is how a Pega application *calls out* to an external REST service — the repair-estimate service, a maps API, a payments gateway.

Key concepts

- **Connect REST rule** — defines the endpoint, HTTP method, headers, and request/response mapping.
- **Authentication profile** — stores how to authenticate (Basic, OAuth 2.0 client credentials, API key, etc.); referenced by the connector so secrets stay out of the rule.
- **Integration classes** — request/response data structures Pega generates to map JSON to/from the clipboard.
- **Data page wrapper** — best practice is to source a **data page** from the connector so the rest of the app consumes clean data, not the raw call.

Create the RepairEstimate Connect REST integration

- 1 In App Studio 'Data' Integrations (or Dev Studio 'Create' Integration-Connectors 'Connect REST') start the **REST integration** wizard.
- 2 Paste the endpoint URL and a **sample request/response** JSON; Pega auto-generates the data model and request/response mapping.
- 3 Configure the **authentication profile** (e.g. OAuth 2.0 client credentials to the estimate provider).
- 4 Pega creates a **data type/data object** and a **data page** (e.g. D_RepairEstimate) bound to the connector.
- 5 Add **error handling**: map non-200 responses, set a status message, and decide retry vs. fail behaviour.
- 6 Consume it: in the Assessment stage, an Automation step (or the data page) calls the service and populates `.RepairEstimates()`.

Connect REST (conceptual)

```
Resource path : https://api.acme-estimates.com/v1/estimates
Method       : POST
Auth profile  : OAuth2-EstimateProvider (client credentials)
Request map   : .Vehicle.Registration -> body.vin
               .DateOfIncident       -> body.incidentDate
Response map  : body.garage           -> .RepairEstimates().Garage
               body.total             -> .RepairEstimates().TotalCost
Wrapped by    : Data Page D_RepairEstimate (read-only, param: VIN)
```

The connector maps clipboard ↔ JSON; a data page gives the app a clean, cacheable interface.

Simulate before the API exists

Pega lets you **simulate** a connector's response so you can build and demo the case flow before the real endpoint is ready. Switch to the live endpoint later without changing the consuming rules.

15. Integration – Service REST (Exposing Pega as an API)

Service REST is the mirror image of Connect REST: it lets external systems *call into* Pega — for example a customer portal creating a claim, or a mobile app fetching claim status.

Key concepts

- **Service Package** (Data-Admin-ServicePackage) — groups services, sets the URL pattern, processing mode and requestor pooling.
- **Service REST rule** — defines the resource path, HTTP method, and maps the incoming request to Pega and the Pega result back to the response.
- **Service activity** — the logic that runs (create a case, run a query) when the endpoint is hit.
- **Authentication** — secure the package with OAuth 2.0 / token / Basic; map to an operator/access group that constrains what the service may do.

Expose a 'Create Claim' REST service

- 1 Create a **Service Package** (e.g. `claims`) and set its authentication type (OAuth 2.0 recommended).
- 2 Create a **Service REST** rule: method **POST**, resource path `/claims`.
- 3 Map the request body to a page of the Claim class; call a **service activity** that runs the standard **pxCreateCase**/case creation to open a new Claim.
- 4 Map the response to return the new caseID and status as JSON.
- 5 Assign an **access group** to the service so it runs with least-privilege rights.
- 6 Test with a REST client (or Pega's **API** tester); confirm a case appears in the work pool.

Constellation DX API vs. your own Service REST

For UI-style access (view/create/update cases and assignments) Pega already provides the **Constellation DX API**, which uses **OAuth 2.0** (with PKCE for browser clients) and needs no per-endpoint privileges. Build your own **Service REST** for bespoke, non-UI integrations that the DX API doesn't cover.

16. Integration – Connect SQL (Custom Database Queries)

When report definitions and Obj- methods aren't enough — complex joins, stored procedures, vendor-specific SQL — **Connect SQL** lets you run explicit SQL against a Pega or external database.

When (and when not) to use Connect SQL

- **Use** for stored procedures, set-based operations, or queries too complex for a report definition.
- **Prefer** report definitions / Obj-Browse / data pages for ordinary reads — they are safer and more portable.
- Always parameterize; **never** concatenate user input into SQL (injection risk).

The pieces

- A **Connect SQL** rule holds SQL blocks keyed by operation: **Open**, **Save**, **Browse**, **Delete**.
- It maps result columns to properties on a step page, just like other connectors.
- It is invoked from an activity using the **RDB-*** methods: RDB-Open, RDB-List, RDB-Save, RDB-Delete.

Query fraud-watch VINs with Connect SQL

- 1 Create a **Database** record for the target schema if it is external.
- 2 Dev Studio ' **Records** ' **SysAdmin** ' **Connect SQL** ' **Create** → **FraudWatchLookup**.
- 3 In the **Browse** tab write parameterized SQL, e.g. `SELECT vin, reason FROM fraud_watch WHERE vin = {Param.VIN}`.
- 4 Map result columns vin/reason to a results page list.
- 5 From an activity call **RDB-List** against FraudWatchLookup, passing the VIN parameter.
- 6 Wrap it in a **data page** so the case flow just reads D_FraudWatch.

Portability & upgrades

Hand-written SQL is database-specific and can complicate upgrades and cloud migrations. Isolate Connect SQL behind data pages, document it, and revisit whether a report definition could replace it later.

17. Data Pages: The Clean Interface to All Data

A **data page** (formerly 'declare page') is an on-demand, self-populating clipboard page. It is the single most important integration/data pattern in modern Pega — and the recommended source for lists, dropdowns and connector results throughout this guide.

Anatomy

Setting	Options / meaning
Structure	Page (single object) or List (many).
Scope	Thread, Requestor, or Node (shared cache scope).
Edit mode	Read-only (cached), Editable, or Savable (can write back).
Source	Data transform, Activity, Connector (REST/SOAP/SQL), Report definition, Lookup, or another data page.
Parameters	Typed inputs that make the page reusable & drive dependent lookups.
Refresh strategy	Reload when older than N, when parameters change, or per when-rule.

Why data pages matter in Constellation

- The Constellation DX API and generated UI expect data to arrive from **data pages**, not ad-hoc activity calls.
- They provide **caching** (node/requestor scope) so reference lists load once and stay fast.
- They enable **cascading dropdowns** (Ch. 8) via parameters, and cleanly wrap **connectors** (Ch. 12–14).
- They decouple *where* data comes from (DB, REST, SQL) from *how* the UI consumes it.

Create a reusable reference data page

- 1 Dev Studio ' **Create** ' **Data Model** ' **Data Page** → **D_ClaimTypes** (List, Node scope, Read-only).
- 2 Source it from a **report definition** or **lookup** on your reference data.
- 3 Set refresh: reload if older than 1 hour (reference data changes rarely).
- 4 Point the **ClaimType** picklist field's list source at **D_ClaimTypes**.

One rule, many callers

Design data pages to be *parameter-driven and reusable*. A single D_CaseById page parameterized by case ID can serve a screen, a report and an API, avoiding duplicated integration logic.

18. The Login Page & Authentication

Every application needs to know *who* is using it. Pega separates **authentication** (proving identity) from **authorization** (what you may do, Ch. 17).

Authentication services

Authentication is configured in **Authentication Service** records (Data-Admin-Authentication). Each defines a mechanism and how the authenticated identity maps to a Pega operator.

Type	Use case
Basic (PRBasic)	Username/password against Pega's own operator store; simplest, good for sandboxes.
SAML 2.0	Enterprise SSO with an identity provider (IdP) such as Okta / Entra ID / PingFederate.
OpenID Connect (OIDC)	Modern SSO over OAuth 2.0; common for cloud IdPs and Constellation.
OAuth 2.0 / Token	Token-based access, especially for APIs and the Constellation DX API.
Custom	Bespoke flows via an authentication activity when the above don't fit.

Authentication for Constellation specifically

When Pega uses **Constellation** as the base application, the **Constellation DX API** always authenticates with **OAuth 2.0**. For browser-based front ends you use the **authorization code flow with PKCE** so the client secret is never exposed in the browser. Unlike the traditional APIs, the Constellation DX API does not require you to create per-endpoint privileges.

Stand up SSO login for ClaimsApp (OIDC example)

- 1 In your IdP (Okta/Entra/etc.) register an application and note the **issuer**, **client ID**, **client secret** and **redirect URI**.
- 2 In Pega, create an **OpenID Connect** or **OAuth 2.0** authentication service; paste the issuer / well-known configuration, client ID/secret and redirect URI.
- 3 Configure **operator provisioning**: map IdP claims (email, groups) to a Pega operator and to an **access group** (so new SSO users land with the right role).
- 4 Set the authentication service on the application's **access group / requestor type** so it becomes the login path for the app URL.
- 5 For the DX API / custom front end, configure the **authorization code + PKCE** flow and register the SPA's redirect URI.
- 6 Test: visiting the app URL should redirect to the IdP, authenticate, and return you to the Constellation portal.

Customising the login experience

- **Branding** — apply your logo, colours and text to the login screen through the login/authentication presentation configuration.

- **Custom login page** — for Basic auth you can override the login markup; for SSO the IdP owns the login screen, so branding happens there.
- **Multi-factor** — enforce MFA at the IdP (recommended) rather than reinventing it in Pega.

Operators, access groups and the login chain

An **Operator ID** record represents a user. It references an **access group**, which references the **application** and one or more **roles**. At login, Pega authenticates the identity, resolves the operator, reads the access group, and thereby determines the application, portal and permissions the session runs with.

Least privilege from day one

Create dedicated access groups (e.g. *Claimant*, *Assessor*, *Manager*) with only the roles each persona needs. Never let real users share the SysAdmin access group used for building.

19. Authorization & Security: Roles, Privileges & Access Control

Once users are authenticated, Pega decides what each may see and do through a layered access-control model combining role-based (RBAC) and attribute-based (ABAC) controls.

The building blocks

Rule	Role in security
Access Group	Binds an operator to an application, portal, roles and RuleSet list.
Access Role Name	A named role (e.g. Claims:Assessor) referenced by access groups.
Access of Role to Object (ARO)	Grants a role CRUD-style permissions on a class (levels 0–5).
Access Deny	Explicitly removes access, overriding grants.
Privilege	A named permission checked by rules/actions (RBAC).
Access When	Conditional access based on data/context (part of ABAC).
Access Control Policy	ABAC: row- and property-level security evaluated at read/update time.

Row and column security (ABAC)

Access Control Policies enforce fine-grained rules — e.g. an assessor may only open claims in their region, and salary/medical fields are masked unless the user holds a privilege. These evaluate on every read/write, so they protect data regardless of which screen or API touches it.

Secure the Claim case

- 1 Create **Access Role Name** records: **Claims:Claimant**, **Claims:Assessor**, **Claims:Manager**.
- 2 For each role, configure **ARO** entries on the Claim class granting appropriate open/modify/delete levels.
- 3 Add **privileges** (e.g. *ApprovePayout*) and require them on the manager approval step.
- 4 Add an **Access Control Policy** restricting claim visibility by region attribute (ABAC).
- 5 Attach the roles to the matching **access groups** (Claimant / Assessor / Manager) from Ch. 16.

Test security as a real persona

Use Pega's '**Access Manager**' and log in (or impersonate) as each persona to confirm users see only what they should. Security bugs are silent until someone finds the data they shouldn't.

20. Reporting: Report Definitions & Insights

Pega's primary reporting rule is the **Report Definition** — a guardrail-compliant, optimized query builder that also powers data pages, dropdown sources and dashboards.

Report definition essentials

- **Columns** — the properties to display; use **exposed** columns (Ch. 5) for performance.
- **Filters** — conditions with parameters; avoid 'ignore case' and unoptimized properties for speed.
- **Joins** — relate classes/tables (keep to a handful; excessive joins hurt performance and trip guardrails).
- **Grouping / aggregation** — summarized reports with counts, sums, averages.
- **Charts** — attach a visualization; surface in dashboards or Constellation Insights.

Build an 'Open claims by assessor' report

- 1 App Studio ' **Reports** (or Dev Studio ' Report Definition) ' **Create** on the Claim class.
- 2 Add columns: ClaimID, ClaimType, ClaimAmount, AssignedTo, Status.
- 3 Filter Status starts with **Open**; add a parameterized filter for assessor.
- 4 Group by AssignedTo; add a **count** and **sum(ClaimAmount)** summary.
- 5 Add a bar chart and save; expose it on the manager dashboard / as an Insight.

Reports are reusable plumbing

A report definition can source a **data page** and therefore a **dropdown** or an **API**. Build the query once and reuse it across UI, integration and analytics.

21. Quality & Delivery: Guardrails, Testing & Deployment

A working app is not a finished app. Pega gives you tooling to keep quality high and to move changes safely from development to production.

Guardrails and the compliance score

Pega continuously checks your rules against best practices and shows a **compliance score**. Warnings are ranked Severe / Moderate / Informational. The recurring themes throughout this guide are exactly the guardrails: prefer **data transforms** over Property-Set-in-activity, avoid **Java** steps, keep activities and flows **modular**, and **optimize** properties you filter on.

Raise your compliance score

- 1 Open **Dev Studio** ' **Configure** ' **Application** ' **Guardrails**.
- 2 Review the **Warning Summary**; sort by severity and fix **Severe** first.
- 3 Refactor Property-Set steps into **Apply-DataTransform**; replace Java with functions.
- 4 Justify any warning you deliberately keep, with a documented reason.

Testing

- **PegaUnit** — automated unit tests for rules (data transforms, decisions, activities) with assertions.
- **Scenario testing** — end-to-end UI tests that walk a case through its life cycle.
- **Tracer & Clipboard** — runtime debugging: watch rule execution and inspect page data.
- **Live UI / DX diagnostics** — inspect the Constellation component tree and the DX API payloads.

Packaging and moving code

- **Lock RuleSet versions** before migrating to freeze what ships.
- **Product rule (RAP)** — defines the RuleSets/data to export as a deployable archive.
- **Branches** — isolate parallel development and merge to the base RuleSet when ready.
- **Deployment Manager** — an automated CI/CD pipeline to promote a product across dev → test → prod stages.

Blueprint & GenAI (Infinity '24/'25)

Modern Pega accelerates the early steps with **Pega GenAI Blueprint** (generate a case life cycle and data model from a plain-language description) and in-product AI assistants. Treat generated output as a strong first draft that you then refine using everything in this guide.

22. Putting It All Together: The ClaimsApp Build Order

Use this as a checklist. It sequences every feature in the guide into one sensible build order for the vehicle-insurance claim application.

- 1 Foundations** — Create the application, organization, RuleSet stack and class hierarchy (Ch. 3).
- 2 Data model** — Build Vehicle and RepairEstimate data objects and the Claim fields; add declare expressions for calculated totals (Ch. 4).
- 3 Storage** — Optimize the properties you'll filter/report on; map any external reference tables (Ch. 5).
- 4 Case life cycle** — Model Intake → Assessment → Decision → Resolved with steps, routing and SLAs (Ch. 6).
- 5 Views** — Configure Constellation views for each step; add conditional visibility (Ch. 7).
- 6 Cascading dropdowns** — Parameterized data pages for Make → Model (Ch. 8).
- 7 Attachments** — Damage-photo category and file capture (Ch. 9).
- 8 Maps** — Geocode the incident address and show a map pin (Ch. 10).
- 9 Data transforms** — pyDefault + SetClaimDefaults for initialization and mapping (Ch. 11).
- 10 Activities** — Only where declarative rules fall short, e.g. CalculatePayout (Ch. 12).
- 11 Decisioning** — RecommendDecision decision table driving the Decision step (Ch. 13).
- 12 Connect REST** — Repair-estimate + geocoding integrations, wrapped in data pages (Ch. 14).
- 13 Service REST** — Expose Create-Claim for the customer portal (Ch. 15).
- 14 Connect SQL** — Fraud-watch VIN lookup where a report definition won't do (Ch. 16).
- 15 Data pages** — Ensure every list/connector is consumed through a clean, cached data page (Ch. 17).
- 16 Authentication** — Configure OIDC/OAuth 2.0 SSO and the Constellation DX API (PKCE) login (Ch. 18).
- 17 Authorization** — Roles, AROs, privileges and access-control policies per persona (Ch. 19).
- 18 Reporting** — Operational reports and dashboards for managers (Ch. 20).
- 19 Quality & delivery** — Fix guardrails, write PegaUnit/scenario tests, package a product, deploy via Deployment Manager (Ch. 21).

Where to learn more (official sources)

- **Pega Academy** (academy.pega.com) — free structured missions and the CSA / CSSA certifications.
- **Pega Community** (community.pega.com) — how-to articles, the Constellation hub and DX component gallery.
- **Pega Documentation** (docs.pega.com) — authoritative reference for every rule type and the DX API.
- **Pega Support** (support.pega.com) — patch calendar, version support and troubleshooting.

Learn by rebuilding this app

The fastest way to internalize Pega is to build ClaimsApp yourself, one chapter at a time, then extend it: add a child RepairApproval case, a mobile channel, and a customer self-service portal fed by your Service REST and DX API.